Build an MVP: High-Accuracy Software Application Name Normalizer

You are a senior Python engineer specializing in data normalization, software identity resolution, web crawling, search systems, and large-scale data processing.

I need you to build a production-quality MVP for a **Software Application Name Normalizer**.

The purpose of this system is to convert messy/raw software application names into **one clean, canonical normalized application name**.

The system will eventually process approximately **907,000+ application records**, so performance, caching, deduplication, deterministic processing, and avoiding unnecessary web searches are extremely important.

---

1. PRIMARY OBJECTIVE

Input:

```
Raw application name
```

Output:

```
Canonical normalized application name
```

The final normalized name must contain ONLY the application/product name.

Do NOT return:

- version numbers
- OS/platform information
- license information
- registration information
- inventory prefixes
- installation metadata
- unnecessary descriptive information
- confidence scores
- explanations
- URLs

The final normalization function should ultimately be capable of returning:

```
"Prion"
```

instead of:

```
"Prion 0.3 (Android)"
```

---

2. IMPORTANT PRINCIPLE

Do NOT blindly remove information.

The system must understand whether information is:

1. installation/version metadata
2. inventory metadata
3. licensing metadata
4. operating-system metadata
5. registration metadata
6. or genuinely part of the official product name

For example:

```
Prion 0.3 (Android)
→ Prion
```

But:

```
Symbolbibliothek Elektrotechnik 2011
→ Symbolbibliothek Elektrotechnik 2011
```

The year `2011` must remain because it is part of the canonical product name.

Therefore:

> NEVER create a rule that blindly removes every four-digit year.

---

3. CANONICAL NAME EXAMPLES

The implementation MUST support examples such as:

```
Microsoft 365
→ Microsoft 365
```

```
Microsoft 365 Apps
→ Microsoft 365 Apps
```

```
Microsoft Office
→ Microsoft Office
```

The system must NOT unnecessarily modify already-canonical names.

General principle:

> If a name is already a valid canonical product name, preserve it exactly except for safe whitespace/unicode cleanup.

For example:

```
Microsoft 365
```

must NOT become:

```
Microsoft
```

or:

```
Microsoft 365 Product
```

or:

```
Microsoft365
```

---

4. NORMALIZATION EXAMPLES

Implement rules capable of handling examples like:

Version removal

```
Prion 0.3
→ Prion
```

```
Prion 0.3 (Android)
→ Prion
```

```
Adobe Reader 11.0
→ Adobe Reader
```

```
Software 2.5.1
→ Software
```

v-prefix versions

```
MapToaster Topo/NZ v11 (reg)
→ MapToaster Topo/NZ
```

```
MapToaster v4.2
→ MapToaster
```

```
Product v12.5
→ Product
```

Registration information

```
MapToaster Topo/NZ v11 (reg)
→ MapToaster Topo/NZ
```

Remove appropriate variants of:

```
(reg)
registered
registration
```

when they represent registration metadata.

Network license

```
Autodesk AutoCAD 2022 Network License
→ Autodesk AutoCAD
```

```
Autodesk SketchBook Pro 2022 Network License
→ Autodesk SketchBook Pro
```

Remove appropriate variants of:

```
Network License
Network Licence
```

when they occur as licensing metadata.

TS prefix

```
TS-LogRhythm Web Console 6
→ LogRhythm Web Console
```

```
TS-Windows Admin Center 1
→ Windows Admin Center
```

Remove `TS-` only when it is an inventory/system prefix at the beginning.

Do NOT remove `TS` when it is genuinely part of the product name.

Games prefix

```
Games - Minecraft 1.20
→ Minecraft
```

Remove `Games` only when it is clearly an inventory/category prefix.

Do NOT alter:

```
Video Games Manager
```

or other cases where Games is genuinely part of the product name.

5. OPERATING SYSTEM INFORMATION

Remove OS/platform information when it represents installation metadata.

Examples:

```
Prion 0.3 (Android)
→ Prion
```

```
Software 2.1 (Linux)
→ Software
```

```
Software 3.0 (Mac)
→ Software
```

Potential OS indicators include:

```
Windows
Windows 10
Windows 11
Mac
macOS
OS X
Linux
Ubuntu
Android
iOS
Unix
```

However, do NOT blindly remove these words.

For example:

```
Windows Admin Center
```

must remain:

```
Windows Admin Center
```

because Windows is part of the actual product name.

---

6. YEAR HANDLING — CRITICAL

Do NOT blindly remove years.

These are examples where the year MUST remain:

```
Symbolbibliothek Elektrotechnik 2011
→ Symbolbibliothek Elektrotechnik 2011
```

If a year appears to be part of the official product name, preserve it.

A year should only be removed when there is strong evidence that it is a release/version identifier.

For example:

```
Some Software 2022
```

may potentially become:

```
Some Software
```

but only if the context/evidence indicates that `2022` is a release/version.

The system must therefore classify years intelligently.

---

7. EXTRA INFORMATION REMOVAL

The system should identify and remove common metadata such as:

```
(reg)
registered
Network License
Network Licence
```

```
trial
evaluation
demo
installer
setup
64-bit
32-bit
x64
x86
portable
for Windows
for Mac
for Linux
```

BUT:

Do not blindly remove every occurrence.

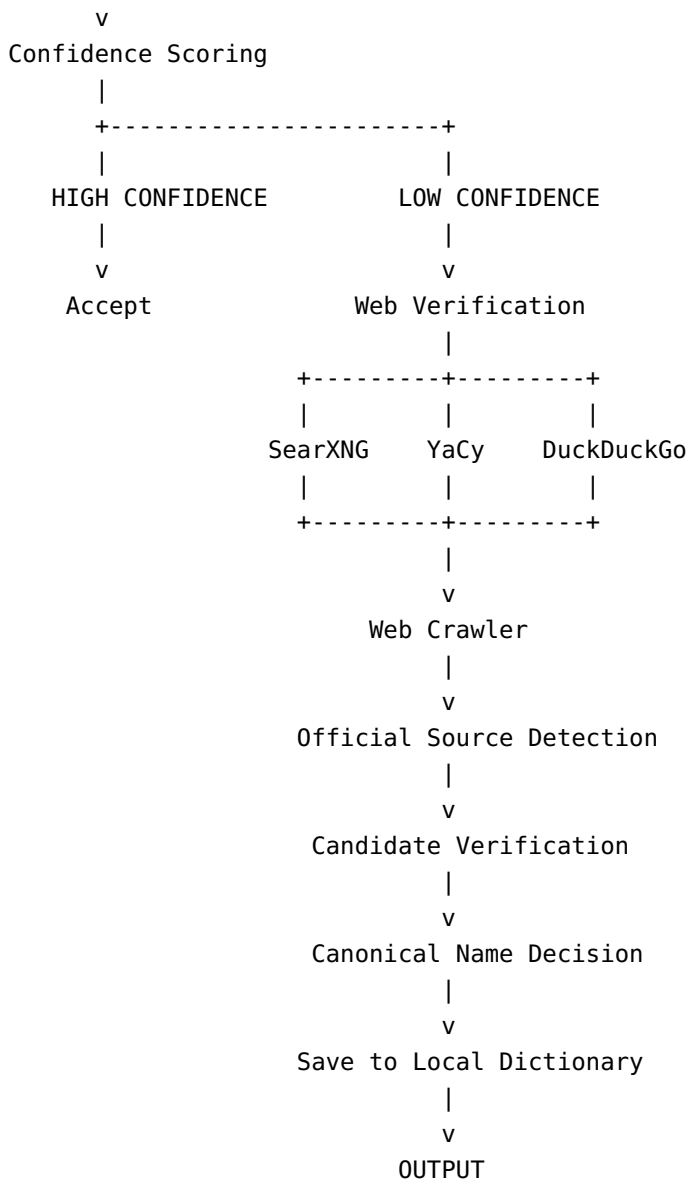
Use contextual rules and candidate validation.

---

8. NORMALIZATION PIPELINE

Implement this architecture:

```
Input CSV/XLSX
      |
      v
Data Loader
      |
      v
Unicode / Whitespace Cleanup
      |
      v
Exact Duplicate Detection
      |
      v
Deterministic Rule Engine
      |
      v
Known Canonical Dictionary
      |
      v
Candidate Generation
      |
```



---

9. SEARCH ENGINES

Implement search clients for:

1. SearXNG
2. YaCy
3. DuckDuckGo

Use them as fallback layers.

DO NOT perform all three searches for every record.

Preferred order:

```
Local dictionary
↓
Deterministic normalization
↓
Known/fuzzy match
↓
SearXNG
↓
YaCy
↓
DuckDuckGo
```

Only proceed to the next search engine if the previous stage cannot establish a reliable answer.

---

10. WEB CRAWLER

Implement a Python crawler that can retrieve and extract useful information from search results.

Use appropriate libraries such as:

```
httpx
BeautifulSoup
selectolax
trafilatura
```

The crawler should:

1. Fetch search-result pages.
2. Extract title.
3. Extract relevant text.
4. Detect likely product/vendor pages.
5. Identify official vendor domains where possible.
6. Extract candidate canonical product names.
7. Avoid unnecessary crawling.
8. Respect robots.txt and reasonable rate limits.
9. Handle timeouts and failed requests gracefully.

Prioritize evidence from:

```
Official vendor websites
Official product pages
Official documentation
Official download pages
Microsoft/Apple/Linux package repositories
GitHub
SourceForge
```

Do not blindly trust random websites.

---

11. CANDIDATE GENERATION

For:

```
MapToaster Topo/NZ v11 (reg)
```

generate candidates such as:

```
MapToaster Topo/NZ
MapToaster Topo/NZ v11
MapToaster Topo/NZ 11
```

Then determine which candidate is the actual canonical name.

For:

```
Prion 0.3 (Android)
```

candidate:

```
Prion
```

For:

```
Symbolbibliothek Elektrotechnik 2011
```

candidate:

```
Symbolbibliothek Elektrotechnik 2011
```

---

12. CONFIDENCE ENGINE

Every normalization decision should internally receive a confidence score.

Example:

```
0.99+ = deterministic/high confidence
0.95-0.99 = strong local evidence
0.85-0.95 = web verified
0.70-0.85 = semantic/weak evidence
<0.70 = manual review
```

The confidence value is for internal processing only.

The final normalized-name output must NOT contain the confidence value.

---

13. LOCAL CANONICAL DICTIONARY

Implement SQLite-based caching.

Create a database containing mappings such as:

```
raw_name
normalized_name
confidence
method
source_url
verified
created_at
updated_at
```

Example:

```
Prion 0.3 (Android)
→ Prion
```

```
TS-LogRhythm Web Console 6
→ LogRhythm Web Console

MapToaster Topo/NZ v11 (reg)
→ MapToaster Topo/NZ
```

Once a name has been successfully verified:

> DO NOT search the web for that exact name again.

Reuse the cached result.

---

14. DUPLICATE OPTIMIZATION

The dataset may contain hundreds of thousands of repeated or similar application names.

Therefore:

```
907,000 rows
```

must NOT result in:

```
907,000 web searches
```

Instead:

```
907,000 rows
      ↓
Unique raw names
      ↓
Normalize unique names
      ↓
Cache results
      ↓
Map results back to original rows
```

This is mandatory.

---

15. FUZZY MATCHING

Use:

```
rapidfuzz
```

for candidate matching.

Detect cases such as:

```
Microsoft 365
Microsoft 365
MICROSOFT 365
Microsoft   365
```

These should resolve to the same canonical name:

```
Microsoft 365
```

But fuzzy matching must NOT incorrectly merge different products.

Use conservative thresholds.

---

16. UNICODE AND TEXT NORMALIZATION

Implement:

- Unicode normalization
- whitespace normalization
- repeated-space removal
- safe punctuation normalization
- case-aware comparison
- Unicode dash normalization
- quotation normalization

However:

> Preserve the canonical capitalization of the actual product name.

For example:

```
Microsoft 365
```

must remain:

```
Microsoft 365
```

not:

```
microsoft 365
```

and not:

```
MICROSOFT 365
```

unless the canonical source itself uses that capitalization.

---

17. RULE ENGINE MUST BE CONFIGURABLE

Do NOT hardcode all rules directly into Python functions.

Create a configuration file such as:

```
config/
    normalization_rules.yaml
    known_exceptions.yaml
    official_domains.yaml
```

Example:

```
prefix_rules:
  - "^TS[-_ ]+"
  - "^Games[-_: ]+"

suffix_rules:
  - "\\s+Network License$"
  - "\\s+Network Licence$"

version_rules:
```

```
  - "\\bv\\d+(\\.\\d+)*\\b"

registration_rules:
  - "\\(reg\\)"
```

Create an exception mechanism for cases where rules must NOT apply.

Example:

```
preserve:
  - "Symbolbibliothek Elektrotechnik 2011"
  - "Microsoft 365"
```

The exception system must be extensible.

---

18. SEMANTIC VERIFICATION

When deterministic rules are uncertain, use an LLM or semantic model only as a verification layer.

The LLM must receive:

```
Raw application name
Candidate normalized name
Search evidence
Official-source evidence
Normalization rules
```

It must decide the canonical product name.

LLM instructions:

```
Return ONLY the canonical application name.

Do not explain your answer.

Do not invent a product.

Do not remove information unless it is clearly metadata.

Preserve years when they are part of the official product name.
```

```
Preserve product editions when they are part of the canonical name.

Remove versions, registration information, license metadata, OS metadata and
inventory prefixes when appropriate.

Prefer official vendor evidence.
```

---

19. FINAL OUTPUT

The primary output should contain:

```
Normalized_Application
```

Example:

```
Prion
MapToaster Topo/NZ
LogRhythm Web Console
Microsoft 365
Symbolbibliothek Elektrotechnik 2011
Autodesk AutoCAD
```

Do NOT put:

```
Confidence
Reason
Method
URL
Search result
Explanation
```

into the final normalized-name column.

These can exist in a separate debug/audit output if required.

---

20. OPTIONAL AUDIT OUTPUT

Create a separate debug file:

```
normalization_audit.csv
```

with:

```
raw_name
normalized_name
confidence
method
rule_applied
search_engine
source_url
verified
```

This is extremely useful for debugging but must not contaminate the final normalized-name dataset.

---

21. PERFORMANCE REQUIREMENTS

The system must be designed for approximately:

```
907,000+ records
```

Requirements:

- Process unique names rather than every row individually.
- Cache all web results.
- Cache normalized mappings.
- Avoid duplicate searches.
- Support batch processing.
- Support resume after interruption.
- Do not restart the entire process if the crawler crashes.
- Use connection pooling where appropriate.
- Use asynchronous HTTP requests where safe.
- Implement retry/backoff.
- Implement rate limiting.
- Store intermediate results.
- Support incremental processing.

The system should be able to stop and resume.

---

22. PROJECT STRUCTURE

Create a clean project:

```
name_normalizer/
│
├── app/
│   ├── __init__.py
│   ├── normalizer.py
│   ├── rules.py
│   ├── candidates.py
│   ├── confidence.py
│   ├── dictionary.py
│   ├── deduplicator.py
│   └── pipeline.py
│
├── search/
│   ├── searxng.py
│   ├── yacy.py
│   └── duckduckgo.py
│
├── crawler/
│   ├── crawler.py
│   ├── extractor.py
│   └── official_source.py
│
├── storage/
│   ├── sqlite.py
│   └── cache.py
│
├── config/
│   ├── normalization_rules.yaml
│   ├── exceptions.yaml
│   └── official_domains.yaml
│
├── input/
│
├── output/
│
├── tests/
│   ├── test_rules.py
│   ├── test_versions.py
│   ├── test_years.py
│   ├── test_os.py
│   ├── test_license.py
│   └── test_examples.py
```

```
│
├── requirements.txt
├── README.md
└── main.py
```

---

23. TEST CASES

Create automated tests for at least these:

```
Prion 0.3 (Android)
→ Prion

MapToaster Topo/NZ v11 (reg)
→ MapToaster Topo/NZ

TS-LogRhythm Web Console 6
→ LogRhythm Web Console

Autodesk AutoCAD 2022 Network License
→ Autodesk AutoCAD

Symbolbibliothek Elektrotechnik 2011
→ Symbolbibliothek Elektrotechnik 2011

Microsoft 365
→ Microsoft 365

Microsoft 365 Apps
→ Microsoft 365 Apps

Windows Admin Center
→ Windows Admin Center
```

Also create negative tests where the normalizer must NOT incorrectly remove words or years.

---

24. IMPORTANT SAFETY AGAINST OVER-NORMALIZATION

The biggest risk in this project is:

> Removing too much information.

For example, this is WRONG:

```
Microsoft 365
→ Microsoft
```

This is WRONG:

```
Symbolbibliothek Elektrotechnik 2011
→ Symbolbibliothek Elektrotechnik
```

This is WRONG:

```
Windows Admin Center
→ Admin Center
```

This is WRONG:

```
Video Games Manager
→ Video Manager
```

The system should prefer:

> Preserve information unless there is strong evidence that it is metadata.

---

25. DEVELOPMENT APPROACH

Do NOT immediately build the 907k-row production pipeline.

First build:

Phase 1 — Core Normalizer

Implement:

- text cleanup
- prefix rules
- suffix rules
- version detection

- OS detection
- registration detection
- license detection
- year handling
- exceptions
- Microsoft 365 preservation
- test suite

Then validate against sample data.

Phase 2 — Dictionary + Fuzzy Matching

Implement:

- SQLite
- caching
- exact matching
- fuzzy matching
- canonical mapping

Phase 3 — Search

Implement:

- SearXNG
- YaCy
- DuckDuckGo fallback

Phase 4 — Crawler

Implement:

- HTTP crawler
- content extraction
- official-source detection

Phase 5 — Semantic Verification

Implement LLM/semantic verification only for uncertain records.

Phase 6 — Large Dataset Pipeline

Implement:

- CSV/XLSX input
- unique-name extraction
- batch processing
- checkpointing

- resume
- final output
- audit output

---

26. INPUT DATA

The supplied Excel file contains examples of application names and existing normalized application values.

Use the supplied dataset to:

1. Analyze existing normalization patterns.
2. Discover recurring prefixes/suffixes.
3. Discover version patterns.
4. Discover license patterns.
5. Discover OS patterns.
6. Discover exceptions.
7. Identify potentially incorrect existing normalized values.
8. Build initial test cases.

IMPORTANT:

Do NOT blindly treat the existing `Normalized_Application` column as ground truth.

Use it as reference/evidence and validate questionable cases.

---

27. EXPECTED DELIVERABLE

Generate the complete working MVP.

Provide:

1. Full project structure.
2. Complete Python source code.
3. YAML configuration files.
4. SQLite schema.
5. Search-engine clients.
6. Web crawler.
7. Candidate generator.
8. Confidence engine.
9. Normalization engine.
10. Excel/CSV processor.
11. Test suite.
12. Requirements file.

13. README with installation instructions.
14. Example configuration.
15. Example input/output.
16. CLI commands for running the system.
17. Resume/checkpoint mechanism.
18. Logging.
19. Error handling.

The MVP must be runnable locally.

---

28. CLI DESIGN

Support commands similar to:

```
python main.py normalize --input input.xlsx --output output.xlsx
```

and:

```
python main.py normalize --input input.csv --output output.csv
```

Also support:

```
python main.py test
```

and:

```
python main.py stats
```

and ideally:

```
python main.py resume
```

---

29. MOST IMPORTANT REQUIREMENT

The system is NOT a generic text cleaner.

It is a:

Therefore the system should combine:

```
Rules
+
Exceptions
+
Existing knowledge
+
Fuzzy matching
+
Search
+
Web crawling
+
Official-source verification
+
Semantic reasoning
+
Caching
```

to determine the most accurate canonical application name possible.

The final objective is:

```
MESSY SOFTWARE NAME
        ↓
        ↓
        ↓
CANONICAL SOFTWARE PRODUCT NAME
```

with **minimum false normalization**.

Prioritize correctness over aggressive cleaning.